

# Exercices Regex en Python

---

## ■ Niveau Facile : Validation et Recherche basique

### Exercice 1 : Le Détective de Chiffres

Écris une fonction `contient_chiffre(texte)` qui renvoie `True` si le texte contient au moins un chiffre, et `False` sinon.

### Exercice 2 : Alerte Majuscules

Écris une fonction `commence_par_majuscule(texte)` qui vérifie si une chaîne commence obligatoirement par une lettre majuscule (A-Z).

### Exercice 3 : Le Code Postal

Écris une fonction `valide_code_postal(code)` qui vérifie si la chaîne passée correspond à un code postal français valide.

Un code postal valide contient exactement 5 chiffres consécutifs.

Exemple : `75001`

### Exercice 4 : Les Chercheurs d'Or — Le mot exact

Écris une fonction `trouve_chat(texte)` qui renvoie `True` si le mot exact `"chat"` est présent.

Attention, elle doit renvoyer `False` pour les mots `"château"` ou `"achat"`.

### Exercice 5 : Nettoyage d'Espaces

Écris une fonction `nettoyer_spaces(texte)` qui remplace tous les blocs d'espaces consécutifs, comme les espaces, tabulations et sauts de ligne, par un seul espace simple.

---

## ■ Niveau Moyen : Extraction et Manipulation

### Exercice 6 : Vol d'Identifiants

Écris une fonction `extraire_hashtags(texte)` qui récupère tous les hashtags dans un tweet.

Un hashtag est un mot commençant par `#`, suivi de lettres, chiffres ou underscores.

Exemple :

```
"J'adore le #Python et le #Deep_Learning !"
```

Résultat attendu :

```
['#Python', '#Deep_Learning']
```

### Exercice 7 : Le Format de Date

Écris une fonction `extraire_dates(texte)` qui trouve toutes les dates au format `JJ/MM/AAAA` dans un texte.

### Exercice 8 : Masquage de Données — RGPD

Écris une fonction `masquer_tel(texte)` qui trouve tous les numéros de téléphone français à 10 chiffres et les remplace par `[REDACTED]`.

Les numéros peuvent être séparés ou non par des espaces, des points ou des tirets.

Exemples :

```
06 12 34 56 78
06-12-34-56-78
0612345678
```

### Exercice 9 : Le piège du HTML — Lazy quantifiers

Soit la chaîne :

```
<p>Premier</p> de l'histoire et <p>Second</p>
```

Écris une fonction `extraire_paragraphes(texte)` qui renvoie la liste du contenu des balises `<p>`.

Résultat attendu :

```
['Premier', 'Second']
```

Attention à ne pas te laisser piéger par le comportement greedy.

### Exercice 10 : Validation de Pseudo de Gamer

Écris une fonction `valide_pseudo(pseudo)` qui valide un pseudo selon ces règles :

- doit commencer par une lettre majuscule ou minuscule ;
- peut contenir des lettres, des chiffres et des underscores `_` ;
- doit faire entre 4 et 12 caractères au total.

Pour ces exercices, la Regex ne fera pas tout le travail.

Tu vas devoir utiliser :

- `re.finditer()`
- `re.sub()` avec des fonctions de callback
- ou boucler sur les résultats pour appliquer une logique algorithmique.

### Exercice 11 : L'Extracteur de Prix — Calculateur

Écris une fonction `total_facture(texte)` qui extrait tous les prix en euros présents dans un texte et renvoie la somme totale sous forme de `float`.

Exemples de prix :

```
12.50€  
49€  
120,75 €
```

Indice : pense à harmoniser les virgules en points en Python avant de convertir en `float`.

### Exercice 12 : Analyseur de logs et IP uniques

Tu reçois un fichier de logs sous forme de chaîne de caractères.

Écris une fonction `obtenir_ip_uniques(logs)` qui extrait toutes les adresses IPv4 valides et renvoie une liste triée des IP uniques.

Une IPv4 valide est composée de 4 blocs de 1 à 3 chiffres séparés par des points.

Exemple :

```
192.168.1.1
```

### Exercice 13 : Le Compresseur de Texte — Run-Length Encoding

Écris une fonction `compresser_texte(texte)` qui trouve les répétitions de lettres consécutives et les remplace par la lettre suivie de son nombre d'apparitions.

Exemple :

```
AAABBCDDDD
```

Résultat attendu :

A3B2C1D4

Indice : utilise `re.sub()` avec une fonction de remplacement, aussi appelée callback, et les groupes capturés.

## Exercice 14 : Valideur de Mot de Passe Complexe

Au tout début de ton cours, il y avait cette regex :

```
r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[-+!*@#$$%^&🍌]).{8,}$'
```

Écris une fonction `evaluer_mot_de_passe(password)` qui n'utilise pas cette regex magique d'un coup.

À la place, utilise 4 ou 5 petites regex simples pour attribuer un score de force sur 5 au mot de passe.

La fonction doit retourner un dictionnaire avec les critères manquants.

## Exercice 15 : Le Markdown vers HTML — Le boss de fin

Écris un mini-parseur Markdown `markdown_to_html(texte)` qui convertit :

- les titres `# Titre` en `<h1>Titre</h1>` ;
- les sous-titres `## Sous-titre` en `<h2>Sous-titre</h2>` ;
- le gras `**texte**` en `<strong>text</strong>` ;
- les liens `[Google](https://google.com)` en `<a href="https://google.com">Google</a>`.

# Jeux de Tests pour la Série d'Exercices Regex

Ce fichier regroupe l'ensemble des cas de tests (nominatifs, limites et négatifs) pour valider tes fonctions Python.



## Script de Test Automatisé (Format Pytest / Python)

```
import re

# =====
# PLACE TES FONCTIONS ICI (ou importe-les)
# =====
# ex: def contient_chiffre(texte): ...

# =====
# ZONE DE TESTS
# =====

# -----
```

```
# NIVEAU FACILE : Validation et Recherche basique
# -----

def test_ex1_contient_chiffre():
    assert contient_chiffre("J'ai 2 chats.") is True
    assert contient_chiffre("Le prix est de 99€.") is True
    assert contient_chiffre("Pas de chiffre ici.") is False
    assert contient_chiffre("") is False

def test_ex2_commence_par_majuscule():
    assert commence_par_majuscule("Bonjour") is True
    assert commence_par_majuscule("A") is True
    assert commence_par_majuscule("bonjour") is False
    assert commence_par_majuscule("123 Bonjour") is False
    assert commence_par_majuscule(" Bonjour") is False # Espace au début

def test_ex3_valide_code_postal():
    assert valide_code_postal("75001") is True
    assert valide_code_postal("06000") is True
    assert valide_code_postal("7500") is False # Trop court
    assert valide_code_postal("750012") is False # Trop long
    assert valide_code_postal("75A01") is False # Contient une lettre
    assert valide_code_postal("abcde") is False # Que des lettres

def test_ex4_trouve_chat():
    assert trouve_chat("Le chat dort.") is True
    assert trouve_chat("Mon chat, il est beau.") is True
    assert trouve_chat("J'ai acheté un château.") is False # Mot composé (début)
    assert trouve_chat("C'est un achat utile.") is False # Mot composé (fin)
    assert trouve_chat("chats") is False # Pluriel (frontière
    brisée)

def test_ex5_nettoyer_espaces():
    assert nettoyer_espaces("Un deux trois") == "Un deux trois"
    assert nettoyer_espaces("Texte\navec\t\tplusieurs espaces") == "Texte avec
    plusieurs espaces"
    assert nettoyer_espaces(" Espace au début et à la fin ") == "Espace au
    début et à la fin"
    assert nettoyer_espaces("Déjà-propre") == "Déjà-propre"

# -----
# NIVEAU MOYEN : Extraction et Manipulation
# -----

def test_ex6_extraire_hashtags():
    assert extraire_hashtags("J'adore le #Python !") == ["#Python"]
    assert extraire_hashtags("#Deep_Learning et #IA") == ["#Deep_Learning", "#IA"]
    assert extraire_hashtags("Pas de hashtag ici.") == []
    assert extraire_hashtags("Le symbole # tout seul") == []
```

```
def test_ex7_extraire_dates():
    texte = "Inscrit le 12/05/2023, modifié le 01/12/2025."
    assert extraire_dates(texte) == ["12/05/2023", "01/12/2025"]
    assert extraire_dates("Date invalide : 123/01/2022") == []
    assert extraire_dates("Autre format : 2026-05-28") == []

def test_ex8_masquer_tel():
    assert masquer_tel("Appelez le 0612345678") == "Appelez le [REDACTED]"
    assert masquer_tel("Fixe : 01.42.56.78.90") == "Fixe : [REDACTED]"
    assert masquer_tel("Mon num: 07-89-10-11-12") == "Mon num: [REDACTED]"
    # assert masquer_tel("International +33 6 12 34 56 78") == "International +33
[REDACTED]"

def test_ex9_extraire_paragraphes():
    html = "<p>Premier</p> de l'histoire et <p>Second</p>"
    assert extraire_paragraphes(html) == ["Premier", "Second"]
    assert extraire_paragraphes("<p></p>") == [""] # Vide mais valide
    assert extraire_paragraphes("<div>Pas de p</div>") == []

def test_ex10_valide_pseudo():
    assert valide_pseudo("Gamer_99") is True # 8 caractere, commence par lettre
    assert valide_pseudo("Alex") is True # Limite basse (4 caractères)
    assert valide_pseudo("X_Ragnarok_X") is True # Limite haute (12 caractères)
    assert valide_pseudo("123Gamer") is False # Commence par un chiffre
    assert valide_pseudo("Ace") is False # Trop court (3 caractères)
    assert valide_pseudo("GamerPolonais99") is False # Trop long (15 caractères)
    assert valide_pseudo("Sniper-45") is False # Caractère interdit (-)

# -----
# NIVEAU DIFFICILE : Regex & Algorithmique
# -----

def test_ex11_total_facture():
    f1 = "Café: 2,50€, Sandwich: 5.50 €, Dessert: 4€."
    assert total_facture(f1) == 12.0
    assert total_facture("Rien n'est payant ici.") == 0.0
    assert total_facture("Gros achat à 1250,99 €") == 1250.99

def test_ex12_obtenir_ip_uniques():
    logs = """
192.168.1.1 - Login success
10.0.0.254 - Bad gateway
192.168.1.1 - Duplicate entry
999.999.999.999 - Fausse IP sauvage
"""
    assert obtenir_ip_uniques(logs) == ["10.0.0.254", "192.168.1.1"]
    assert obtenir_ip_uniques("Aucune IP") == []
```

```
def test_ex13_compresser_texte():
    assert compresser_texte("AAABBCDDDD") == "A3B2C1D4"
    assert compresser_texte("A") == "A1"
    assert compresser_texte("AABBBBA") == "A2B4A1" # Répétitions séparées
    assert compresser_texte("") == ""

def test_ex14_evaluer_mot_de_passe():
    # Test mot de passe parfait (5/5)
    r1 = evaluer_mot_de_passe("Secr3t!_")
    assert r1["score"] == 5
    assert len(r1["manquants"]) == 0

    # Test mot de passe faible (2/5)
    r2 = evaluer_mot_de_passe("bobby")
    assert r2["score"] == 2 # Longueur non validée, pas de maj, pas de chiffre,
    pas de spécial
    assert "Au moins une majuscule" in r2["manquants"]
    assert "Au moins un chiffre" in r2["manquants"]
    assert "Au moins 8 caractères" in r2["manquants"]

def test_ex15_markdown_to_html():
    md = "# Titre\n## Sous-titre\nDu **gras** et un [lien](http://test.com).\"
    attendu = "<h1>Titre</h1>\n<h2>Sous-titre</h2>\nDu <strong>gras</strong> et un\n<a href=\"[http://test.com](http://test.com)\">lien</a>.\"
    assert markdown_to_html(md) == attendu

if __name__ == "__main__":
    print("Pour lancer les tests, installez 'pytest' et exécutez la commande :
    pytest <nom_du_fichier>.py")
```